

Đặc tả: Đồng bộ dữ liệu sinh viên (Legacy CSV Data Sync)

1. Mô tả

Hệ thống quản lý đào tạo hiện tại của trường chưa cung cấp API để tích hợp trực tiếp. Cách duy nhất để lấy danh sách sinh viên hợp lệ là thông qua một file CSV được hệ thống cũ tự động xuất (export) ra ổ cứng máy chủ vào ban đêm. Tính năng này là một tiến trình chạy ngầm (Cronjob Worker), có nhiệm vụ định kỳ đọc file CSV này, làm sạch dữ liệu và cập nhật (Upsert) vào bảng Users trên PostgreSQL. Mục tiêu là tự động tạo tài khoản cho sinh viên để họ có thể đăng nhập vào UniHub vào sáng hôm sau mà không cần phải tự đăng ký tài khoản.

2. Luồng chính (Happy Path)

Bối cảnh: 02:00 AM mỗi đêm, thời điểm hệ thống UniHub có ít lượt truy cập nhất. Hệ thống Legacy đã xuất thành công file students_YYYYMMDD.csv vào thư mục /data/import.

B1. Trigger (Kích hoạt): Hệ thống sử dụng tính năng **Repeatable Jobs** của thư viện **BullMQ (Redis)** để cấu hình một tác vụ chạy ngầm định kỳ vào lúc 02:00 AM (Cron expression: 0 2 * * *), tự động đánh thức Worker.

B2. Kiểm tra File (File System Check): Worker quét thư mục /data/import tìm file CSV mới nhất.

B3. Đọc dữ liệu dạng luồng (Streaming): Để tránh tràn RAM (Out of Memory) khi file CSV có dung lượng lớn (ví dụ chứa 30.000 sinh viên), Worker **KHÔNG** đọc toàn bộ file vào bộ nhớ. Thay vào đó, sử dụng hàm fs.createReadStream() của Node.js kết hợp thư viện parse CSV để đọc dữ liệu theo từng dòng (Chunk/Stream).

B4. Làm sạch & Chuẩn hóa (Data Cleansing): Tại mỗi dòng được đọc:

- Kiểm tra định dạng email (phải có đuôi @student.truong.edu.vn).
- Chuẩn hóa full_name (Xóa khoảng trắng thừa, viết hoa chữ cái đầu).

B5. Xử lý hàng loạt (Batch Processing & Upsert): Thay vì Insert từng dòng (gây chậm DB), Worker sẽ gom các dòng hợp lệ thành từng “Lô” (Batch), ví dụ 1.000 bản ghi/Batch.

Worker thực hiện câu lệnh **UPSERT** của PostgreSQL: `INSERT INTO Users (email, full_name, password_hash, role) VALUES (...) ON CONFLICT (email) DO UPDATE SET full_name = EXCLUDED.full_name.` (Lệnh này có nghĩa là: Nếu email chưa có thì Thêm mới, nếu email đã có thì Cập nhật lại tên).

B6. Lưu trữ & Báo cáo (Archive & Notify): Sau khi đọc hết file, Worker di chuyển file CSV đó sang thư mục `/data/archive` và đổi tên thành `students_YYYYMMDD_DONE.csv` để tránh đọc lại vào hôm sau.

Ghi Log hệ thống: “Đã đồng bộ thành công X bản ghi, thất bại Y bản ghi”.

3. Kịch bản lỗi (Edge Cases & Exception Handling)

3.1. File CSV không tồn tại hoặc sai tên định dạng

- **Trigger:** Đến 02:00 AM nhưng hệ thống cũ bị lỗi nên không xuất ra file CSV, hoặc đổi tên file sai quy chuẩn.
- **Xử lý:** Worker quét không thấy file. Ghi một cảnh báo (Warning) vào log: “Không tìm thấy file CSV đồng bộ ngày YYYYMMDD”. Tác vụ kết thúc an toàn, không gây crash hệ thống.

3.2. Dữ liệu rác hoặc thiếu trường quan trọng (Dirty Data)

- **Trigger:** Trong file CSV có những dòng bị lỗi phong chữ, thiếu cột email, hoặc sai định dạng.
- **Xử lý:** Áp dụng nguyên tắc **Skip & Continue**. Khi bước B4 phát hiện một dòng lỗi, Worker KHÔNG DỪNG toàn bộ tiến trình. Nó sẽ ghi dòng dữ liệu lỗi đó kèm lý do vào một file `error_YYYYMMDD.log` riêng biệt, sau đó tiếp tục xử lý các dòng tiếp theo. Đảm bảo 99% sinh viên hợp lệ vẫn được đồng bộ bình thường.

3.3. Đang đồng bộ thì Worker bị Crash (Mid-process failure)

- **Trigger:** Đang xử lý đến dòng thứ 15.000 thì máy chủ bị cúp điện hoặc Worker bị khởi động lại.

- **Xử lý:** Nhờ sử dụng cơ chế **UPSERT** ở bước B5 (chống trùng lặp theo Email), khi Worker khởi động lại và đọc lại file CSV từ đầu, 15.000 dòng đầu tiên sẽ chỉ thực hiện hành động Cập nhật (Update) chứ không sinh ra dữ liệu rác (Duplicate records). Tính toàn vẹn dữ liệu được bảo vệ tuyệt đối.

3.4. Hệ thống Database đang quá tải (Database Bottleneck)

- **Trigger:** Nếu ban đêm tình cờ có lượng sinh viên thức khuya truy cập lớn, việc nhồi 1.000 bản ghi/lần có thể làm Database chậm đi.
- **Xử lý:** Giữa các lần đẩy Batch (1.000 bản ghi), Worker sẽ áp dụng một độ trễ nhỏ (Sleep 100ms - 200ms) để “nhường” CPU và Connection cho các API xử lý request của sinh viên. (kỹ thuật Throttling the Background Job).

4. Ràng buộc (Constraints)

- **Quản lý Bộ nhớ (Memory Constraint):** Lượng RAM tiêu thụ của Node.js Worker khi đọc file CSV không được vượt quá 100MB, bắt buộc dung lượng file CSV là 10MB hay 1GB (Bắt buộc phải dùng Stream).
- **Tính độc lập (Isolation):** Tiến trình đồng bộ CSV phải chạy trên một Thread/Process riêng biệt (hoặc 1 container Docker riêng), tuyệt đối không dùng chung luồng thực thi (Event Loop) với Core Backend API để không làm chặn (Block) các request HTTP của người dùng.
- **Tính Lũy đẳng (Idempotency):** Việc chạy file CSV đồng bộ 1 lần, hay chạy lại file đó 10 lần, đều phải ra cùng một kết quả duy nhất trong Database (Không sinh ra dữ liệu trùng lặp).
- **Phạm vi đồng bộ (Sync Scope):** Trong giới hạn của hệ thống hiện tại, tiến trình CSV Sync chỉ hỗ trợ cơ chế Add/Update (Thêm mới sinh viên hoặc Cập nhật thông tin). Việc xử lý Deactivate (Khóa tài khoản sinh viên đã nghỉ học) nằm ngoài phạm vi của tiến trình đồng bộ tự động này và sẽ được Admin xử lý thủ công nếu cần.

5. Tiêu chí chấp nhận (Acceptance Criteria)

- **Test Case 1 (Streaming & Performance):** Nạp một file CSV chứa 50.000 bản ghi (khoảng 10MB). Kích hoạt Worker. Quá trình xử lý không văng lỗi Out of Memory và Database có đúng 50.000 tài khoản STUDENT mới.

- **Test Case 2 (Idempotency / Upsert):** Chạy lại chính xác file CSV của Test Case 1 lần thứ hai. Số lượng bản ghi trong Database giữ nguyên không đổi (50.000), không có lỗi văng ra.
- **Test Case 3 (Fault Tolerance):** Tạo một file CSV có 1.000 dòng chuẩn, xen kẽ 5 dòng bị cố tình làm sai định dạng email. Chạy Worker. Kết quả Database nhận đúng 1.000 user. File error.log sinh ra ghi nhận chính xác dòng số bao nhiêu bị lỗi và lý do lỗi.
- **Test Case 4 (Archiving):** Sau khi chạy thành công Test Case 1, kiểm tra lại thư mục /data/import phải trống, và file CSV đã được chuyển an toàn sang /data/archive với hậu tố _DONE.